Introduction to the Unified Modeling Language (UML)

Part 1 - EX-P1

Marcello Missiroli & Giancarlo Succi



UML OOP Bases



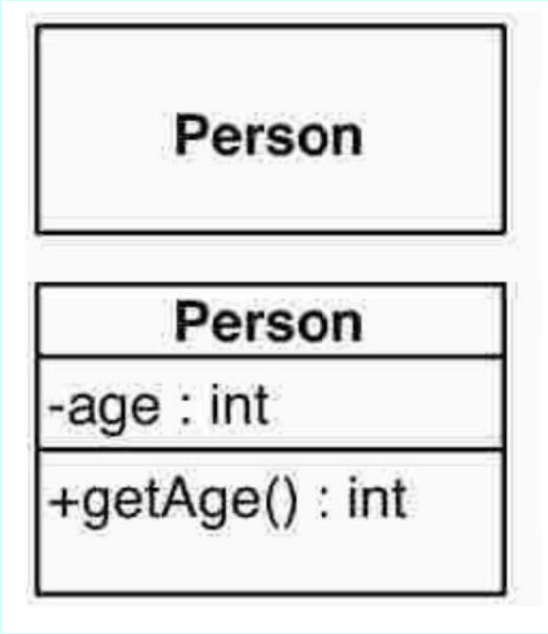
UML Class Icon

- A class has a graphical representation in the form of a rectangle divided into up to three parts or slots.
- – **the name and optional stereotype** (in UpperCamelCase – mandatory)
- – **attributes/properties** (in lowerCamelCase – optional)
- – **operations/methods** (in lowerCamelCase – optional)
- The same class may appear with different amounts of decorations in different diagrams.



Classes in UML

Both examples are legal UML



Attributes & Methods

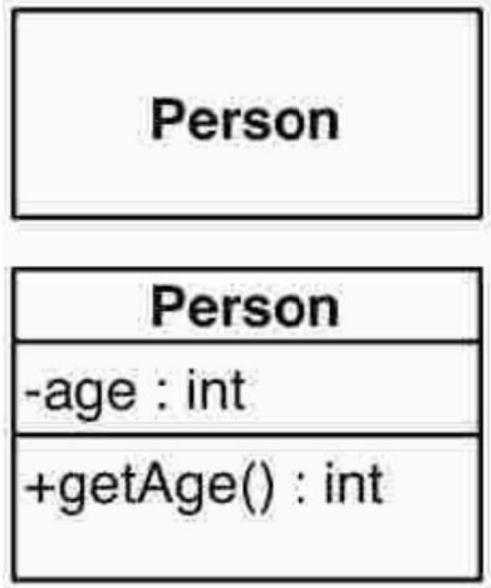
Attribute format:

```
visibility name multiplicity:Type =
initialValue
```

Operation format:

```
visibility name
(parameterName:parameterType, ...) :
returnType
```

Only the name is mandatory.





Attributes and Operations: Visibility Types

- **+**

Public : Every element that can access the class can also access all of its public members.

- **-**

Private: Only the operations of the class itself can access its private members.

- **#**

Protected: Only the operations belonging to the class or its descendants can access protected members.

- **~**

package: Any element in the same package as the class (or in one of its nested sub-packages) can access the class's package-visible members.



UML Object icon

Instances of classes (objects) have a very similar notation to classes.

The title of an object is **underlined** and follows the format:

name : ClassName

(with the name being optional).

Objects do not have a slot for operations,

but they can define values for their attributes.

UML Object icon



UML Object icon

Instances of classes (objects) have a very similar notation to classes.

The title of an object is **underlined** and follows the format:

name : ClassName

(with the name being optional).

Objects do not have a slot for operations,

but they can define values for their attributes.

Class-object relationship

This is represented as a **dependency** with the stereotype «instanceOf»
(the object “instantiates” the class).

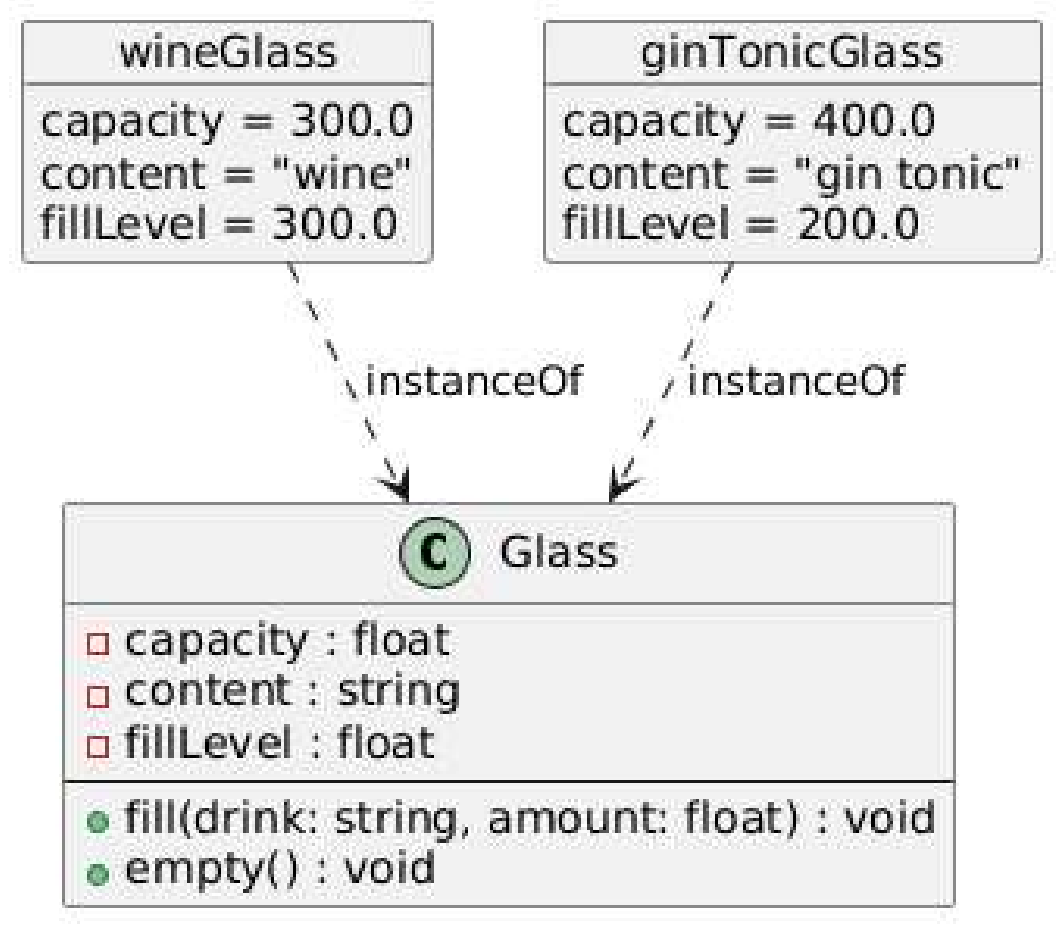




Exercise 1

- Represent in UML a class called “Glass”, with at least 3 attributes and 2 methods. Then instantiate 2 glasses, one full of wine, the other half full of gin tonic

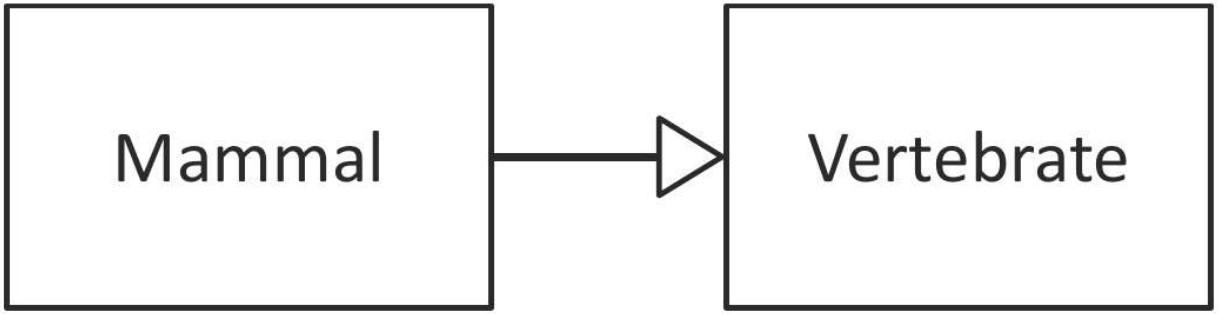
Exercise 1 – possible solution



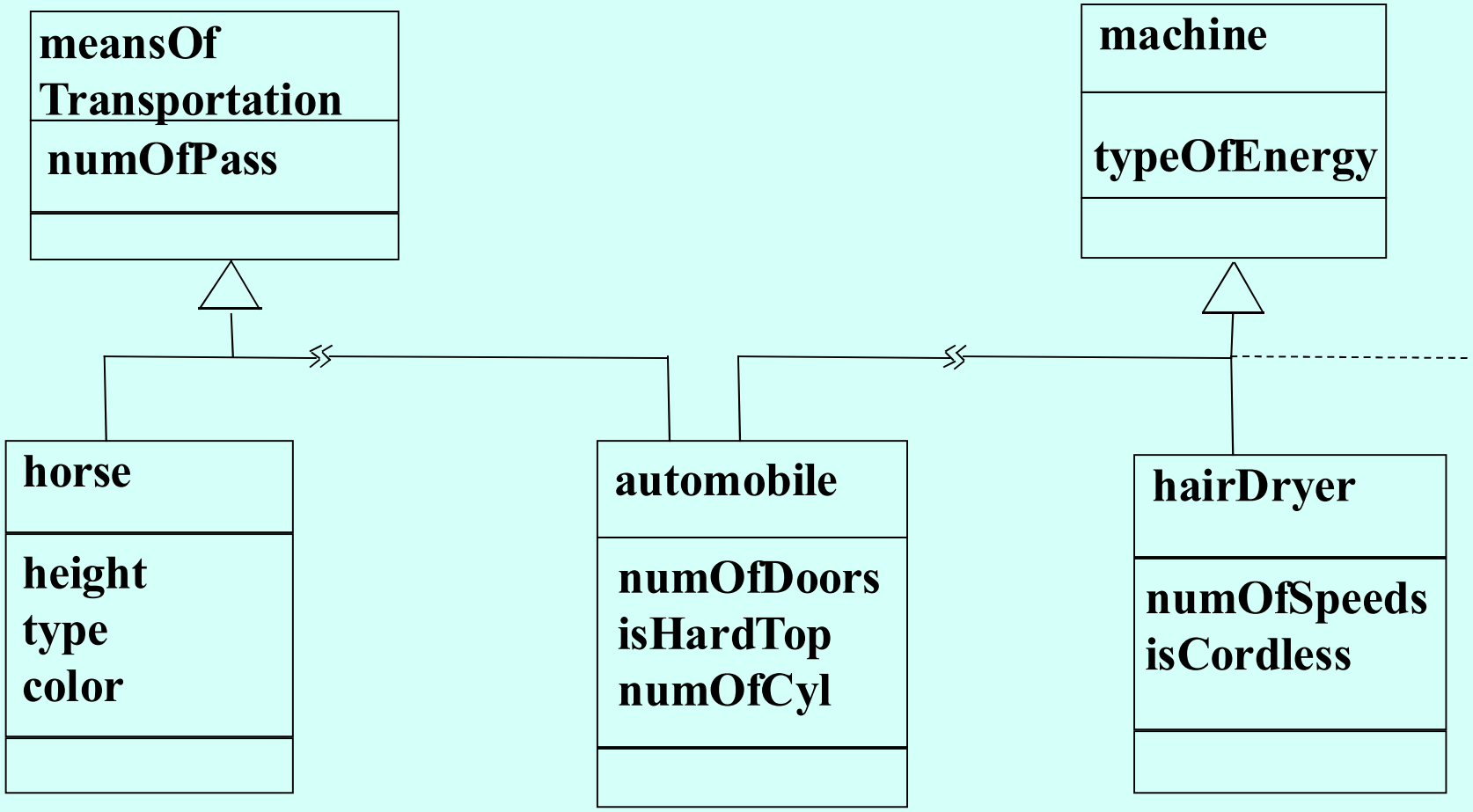


Inheritance in UML

- Inheritance is represented by the **Generalization** relationship in UML.
- The arrow starts from the specific (child) element and points to the more general (parent) element.
- Among all UML relationships, this is the strongest and most constraining.



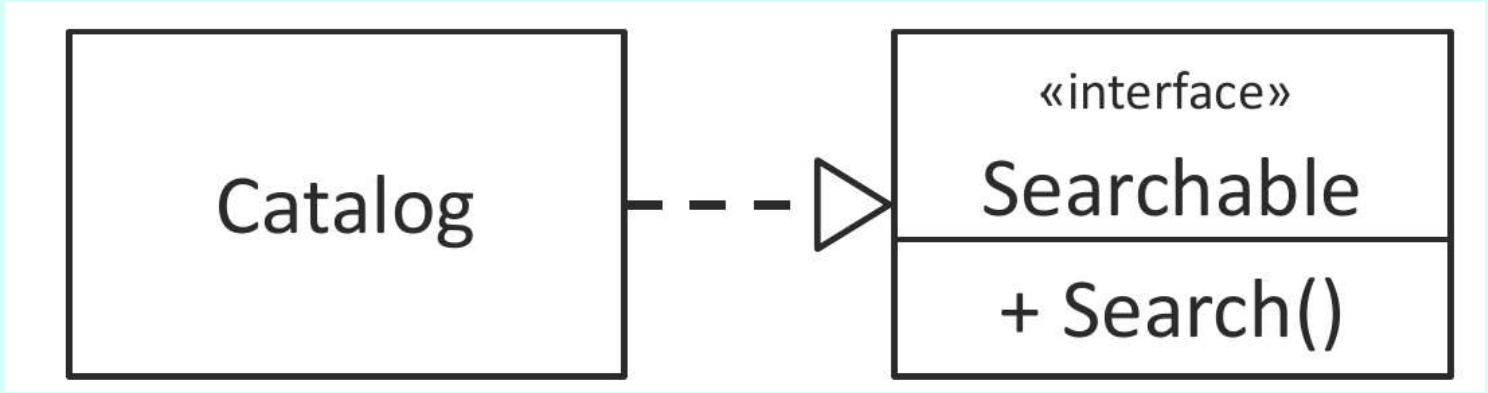
Multiple Inheritance





Realization relationship

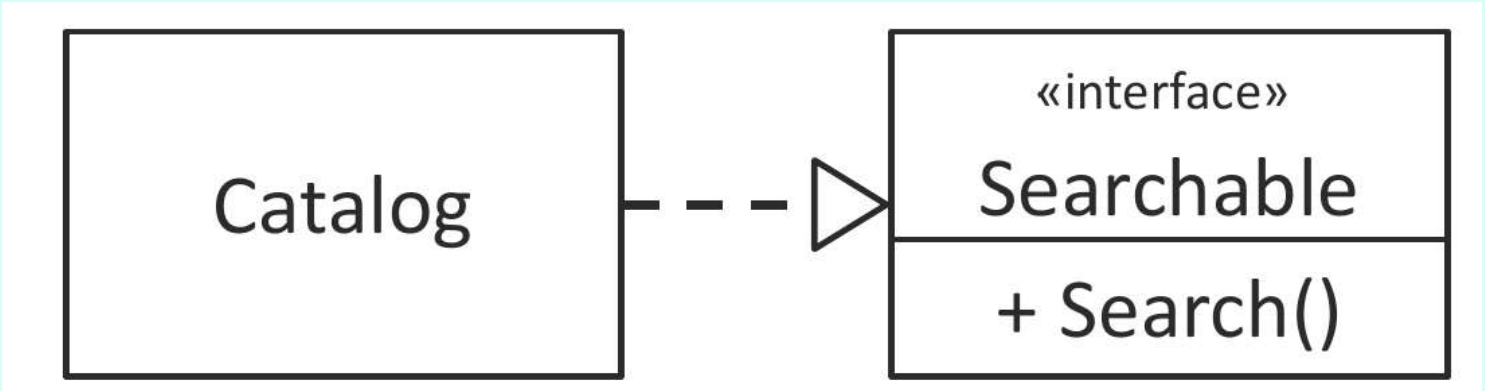
- A semantic relationship in which the supplier provides a specification, and the client realizes it (by implementing and executing it).
- The canonical example of a realization is when the supplier is a Java-style interface, and the client is the class that implements it.





Realization relationship

- A semantic relationship in which the supplier provides a specification, and the client realizes it (by implementing and executing it).
- The canonical example of a realization is when the supplier is a Java-style interface, and the client is the class that implements it.





Abstract classes and methods

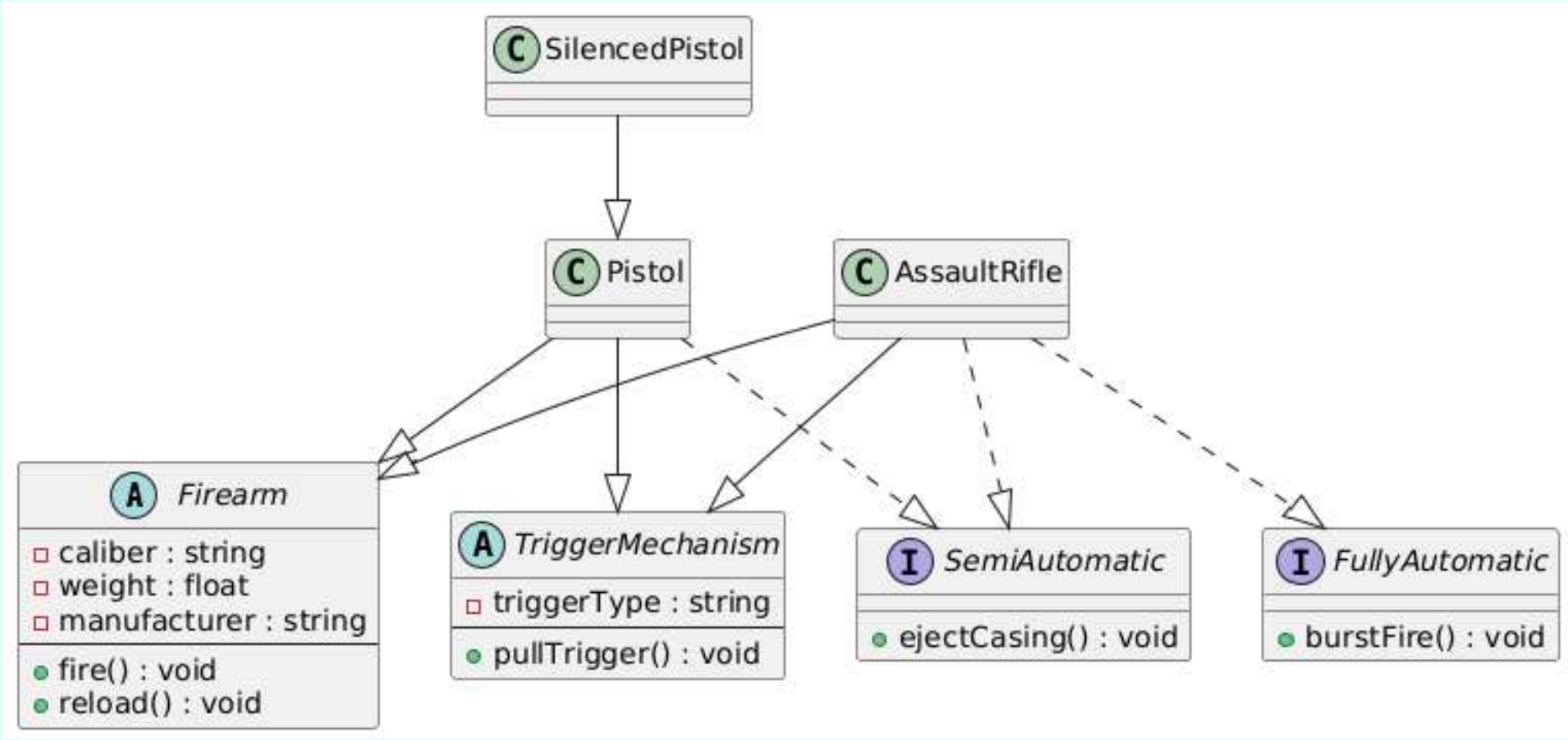
- Abstract classes and methods are written in *italic*
- However, since this is difficult to represent and notice in some cases you can
  - Use the <<abstract>>
  - Mark the constructor as protected (#)
  - Annotate methods as <abstract>



Exercise 2

- Design a UML class diagram for a system that models firearms and their firing mechanisms. Begin with an **abstract class Firearm**, which includes attributes like caliber, weight, and manufacturer, and declares an **abstract method fire()**, along with **a concrete reload()** method.
- Include an **abstract class TriggerMechanism** with a **triggerType attribute and an abstract method pullTrigger().**
- Add **two interfaces**: **SemiAutomatic** with the method ejectCasing(), and **FullyAutomatic** with the method burstFire().
- Then, model a **Pistol** that inherits from Firearm, uses a trigger mechanism, and implements SemiAutomatic. An **AssaultRifle** should also inherit from Firearm, use a trigger mechanism, and implement both interfaces. A SilencedPistol is a special type of Pistol

Exercise 2 - Solution





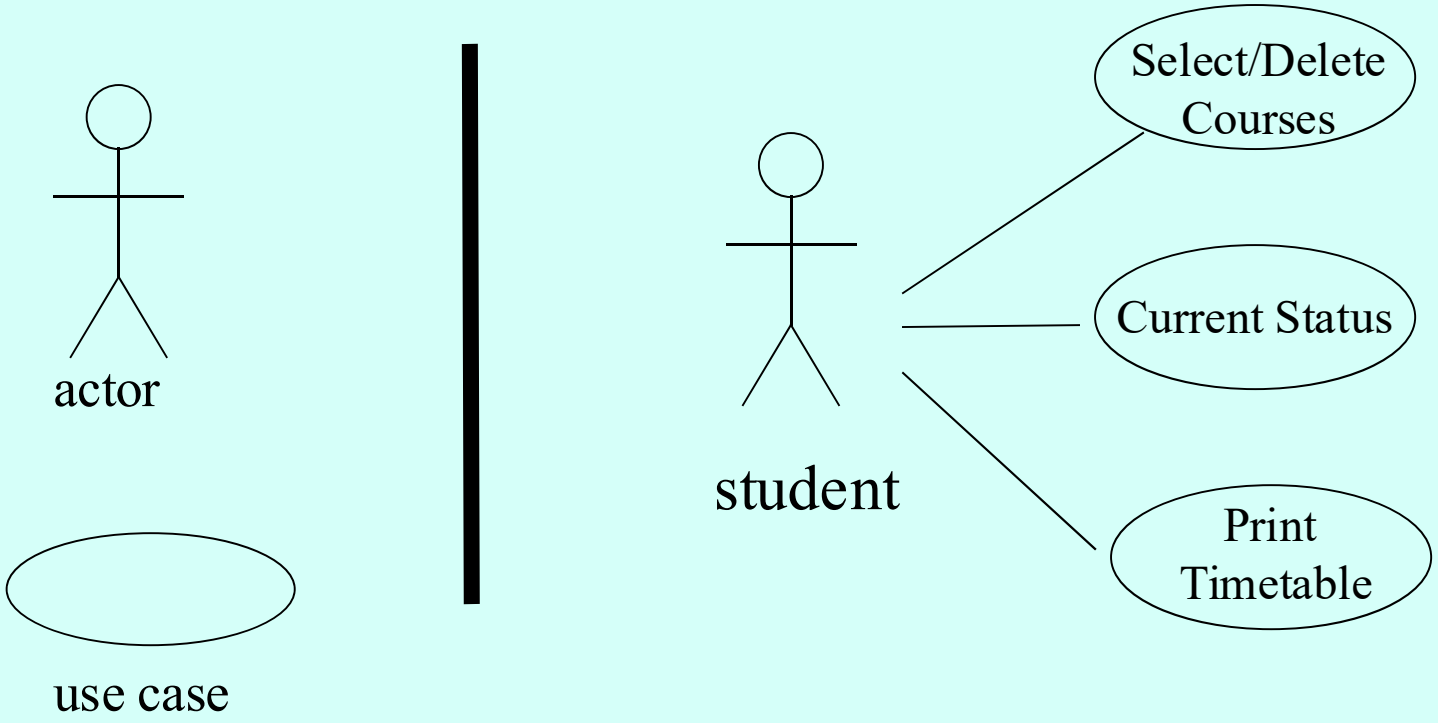
Use Cases Diagram



Use Cases diagram

- A use case diagram expresses a behavior, either desired or offered.
- The subject under examination is usually a system or a part of it.
- It identifies:
  - Who or what interacts with the system (actor)
  - What the actor can do (use case)
- It is typically the first type of diagram created in a development process,as part of requirements analysis.

Use Case Diagram - Early Example





Actors

- An actor specifies a role played by a user or another entity that interacts with the system within the scope of a use case.
- An actor is external to the system.
- It is not necessarily human — it can be a physical object, a software agent, or environmental conditions, etc.
- Actors perform use cases:
- First, identify the actors, then identify their associated use cases.



How to identify actors?

- Who/what uses the system?
- What role does that entity play?
- In which part of the organization is the system used?
- Who/what starts the system?
- Who will support and maintain the system?
- Do other systems interact with this one?
- Are there periodic functions? (e.g., backups)
- Who/what provides or receives information from the system?
- Does an actor have multiple roles?
- Is the same role assigned to multiple actors?



Use case

- A use case is the **specification of a sequence of actions,** including any alternative and/or error sequences, that a system (or subsystem) can execute while interacting with external actors.
- A use case describes something the actor wants the system to do:
  - It is described from the actor's point of view
  - It is triggered by an actor's action
  - The name (label) should be based on a verb or a noun expressing an event.



Use Case: goal

- A use case specifies **what** is expected from a system.
- It **hides how** the system implements that behavior.
- It is a sequence of actions (with possible variants) that produces a result observable by an actor.
- Use cases are used to:
  - Describe requirements (early analysis)
  - Validate the architecture and verify the system



Use Case

- A use case is always initiated by an actor: in UML, an event is always linked to the entity that generated it.
- The actor that initiates the use case is called the primary actor
- Other actors involved during the execution are called secondary actors
- A use case is a classifier with behavior:
  - It can be specified with state diagrams, interaction diagrams, sequence diagrams, and can include preconditions and postconditions.
  - It may also contain variations from the main flow of behavior.



How to identify use cases?

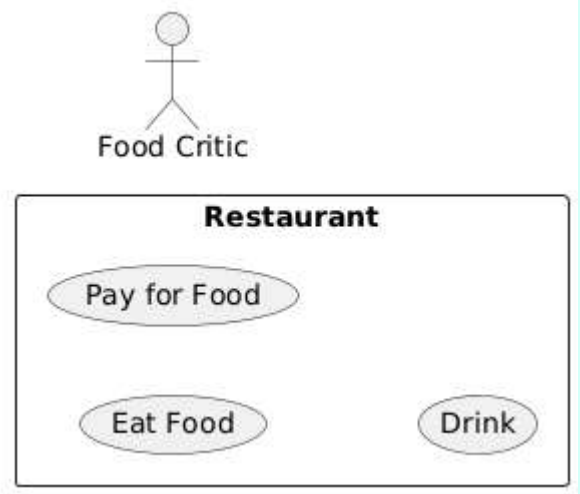
- What functions does each actor expect?
- Does the system manage information (store/provide)?

→ Which actors trigger this?

- Are any actors notified when the system state changes?
- Do actors need to notify the system of sudden changes?
- Do any external events affect the system?
- Which use cases relate to system maintenance?
- Are all functional requirements covered by use cases?

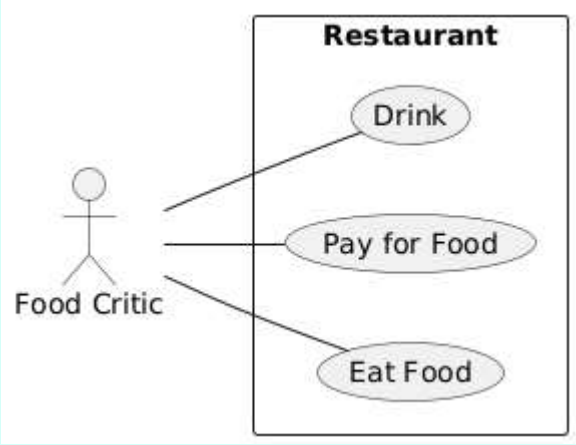
The System

- A simple box enclosing the system
- Actors are always external to the system



Associations

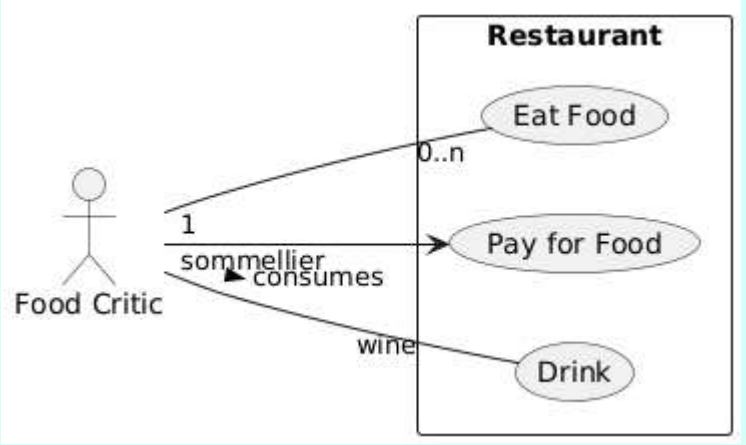
- An association connects actors to use cases.
- An actor can only be associated with use cases, classes, or components (which in turn may be associated with other use cases, classes, and components).
- A use case should not be associated with other use cases that concern the same subject.



Associations

Optional features common to all associations in UML:

- Name
- Multiplicity
- Roles





Easy exercise (with pitfalls)

Consider the definition of a dice game called **Stupid Craps.**

The player places a bet on a number between 1 and 6. The player then rolls a die. If the chosen number appears, the player wins five times the original stake.

Task:
Draw a Minimal UML Use Case Diagram for this system.
You have 3 minutes to complete the exercise.
The work may be done individually or in pairs.

Spoiler?

Easy exercise (solution)

Just one Actor and some UC
No sequentiality
No dice
No classes
No conditionals

Did you overdo??

